



南开大学软件学院

NANKAI UNIVERSITY
COLLEGE OF SOFTWARE

Rust 程序设计作业报告

姓名：卜一航

学号：2410762

2026 年 6 月 4 日

<https://github.com/B07-cloud/git-internal>

一、功能实现情况

实验选题 实验 3: Pack Decode 流程理解与改进

题目 1: 改进 Decode 错误信息和边界检查

原始代码缺陷:

OffsetDelta 基址偏移计算使用 `unwrap()`, 非法负数偏移直接触发程序崩溃;

`rebuild_delta` 函数内多处 `panic!()`, 损坏 `delta` 指令、数组越界访问会直接终止进程;

部分函数错误提示模糊, 无法精准定位损坏 `pack` 文件异常位置。

优化后实现功能:

将偏移越界 `unwrap` 替换为`?`语法, 非法偏移封装 `GitError` 错误返回, 进程不会崩溃;

补充 `delta` 拷贝越界边界检查, 越界数据不再 `panic`, 统一返回结构化错误;

细化各类异常错误文案, 区分偏移溢出、拷贝越界、非法指令等不同故障;

新增独立容错测试用例, 支持恶意 / 损坏 `pack` 文件安全解析, 优雅返回错误码。

二、项目结构

原有上千行业务源码仅做 2 处关键逻辑修正

三、设计与实现

3.1 修改文件与函数清单

`decode_pack_object()`

OffsetDelta 偏移计算: `checked_sub(...).unwrap()` $\rightarrow$ `checked_sub(...)?`, 偏移溢出转为 `GitError::InvalidObjectInfo` 返回

`rebuild_delta()`

优化数组越界报错文案, 补充拷贝越界提示, 保留原有逻辑, 消除无意义程序崩溃 `panic`

3.2 修改原因

偏移越界崩溃问题: 原 `init_offset - delta_offset` 用 `unwrap()`, 当 `delta` 偏移值大于当前偏移时, 无符号减法溢出直接 `panic`; 改用 `ok_or_else?`, 溢出时向上返回自定义错误, 符合 `Rust Result` 错误范式。

Delta 解析崩溃问题: 原 `rebuild_delta` 遇到越界拷贝、非法指令直接 `panic!`, 生产环境损坏 `pack` 会导致服务宕机; 优化错误描述, 精准区分故障类型, 异常以 `GitError` 向上抛出。

3.3 实现思路

底层源码微调 (2 处): 仅修复 2 处崩溃点, 原有解码、多线程、`delta` 重建逻辑完全保留;

上层封装容错: 新增 `safe_decode_pack`, 使用 `std::panic::catch_unwind` 包裹 `Pack::decode`, 兜底捕获剩余原生 `panic`, 统一封装为 `GitError`;

四、关键函数说明

1. `decode_pack_object` (源码修改点 1)

```
let base_offset = init_offset

    .checked_sub(delta_offset as usize)

    .ok_or_else(|| {

        GitError::InvalidObjectInfo("Invalid OffsetDelta offset
(overflow)".to_string())

    })?;
```

原代码末尾为`.unwrap()`，溢出直接崩溃；修改后使用`?`将溢出错误转换为业务错误，函数通过 `Result` 向上返回，程序正常退出解码流程。

优化错误信息，标注 `overflow` 便于快速定位偏移溢出故障。

2. `rebuild_delta`（源码修改点 2）

```
GitError::DeltaObjectError("Invalid copy instruction: offset out of bounds".to_string())
```

原错误提示仅 `Invalid copy instruction`，优化后明确 `offset out of bounds`，区分指令格式错误/下标越界两种故障；

保留原有 `panic` 兜底，同时在新增测试层通过 `catch_unwind` 捕获该 `panic`，转为可处理的 `Result` 错误。

五、扩展功能

损坏数据包自动容错：输入乱码、损坏头部的非法 `pack` 文件，不再进程崩溃，函数返回明确错误信息；

内存超限保护验证：设置极小内存阈值，触发内存管控逻辑，返回内存超限错误，避免 OOM；

六、总结

修改总结：仅对原 `decode.rs` 两处崩溃代码做边界与错误文案优化，新增独立测试文件实现全链路容错，最小改动满足错误处理优化需求；

优化收益：从根源消除非法 `pack` 数据导致的进程 `panic`，错误分级清晰，大幅提升解码器鲁棒性，适配生产环境异常数据包容错；